

# MULTI-QUBIT SYSTEM

## Tensor Product

Single qubit states live in a Hilbert Space, which is a 2-dimensional vector space spanned by basis vectors  $|0\rangle$  and  $|1\rangle$

**Hilbert Uzayı;** Kuantum mekaniğinde kullanılan soyut, matematiksel yapı. Sonsuz boyutlu olabilir, ancak tek qubit için 2 boyutlu olan bir vektör uzayı. Bu uzay  $\rightarrow$  kuantum durumlarının matematiksel olarak temsil edildiği yer.

**Baz vektörleri;** Bu iki boyutlu uzayın baz vektörleri  $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ ,  $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$  kuantum mekaniğinde bir qubit'in iki olası durumunu ifade eder.

**Hilbert Uzayındaki Durumlar;** Bir kuantum durumu, bu baz vektörlerinin doğrusal kombinasyonu olarak ifade edilir.  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

Bu genlikler **Born Kuralına Göre** kuantum durumunun ölçüm olasılıklarını verir ve bu koşulu sağlar;  $\left\{ \begin{array}{l} \text{Kuantum durumunun genliklerini temsil} \\ \text{eden karmaşık sayılar.} \end{array} \right.$

$$|\alpha|^2 + |\beta|^2 = 1$$

**Block Sphere;** Kubit'in durumu, kürenin yüzeyinde bir nokta olarak gösterilir ve bu noktaya, iki baz vektörün ( $|0\rangle$  ve  $|1\rangle$ ) bir kombinasyonudur.

When we put more than one qubit together  $\rightarrow$  how these vector spaces are composed.  
Hilbert Spaces are combined using  $\rightarrow$  **Tensor Product**

$$\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} a \begin{pmatrix} c \\ d \end{pmatrix} \\ b \begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}$$

Sadece Quantum State'ler için geçerli değil.  
Aynı zamanda bu durumlar üzerinde işlem yapan unitary işlemler için de geçerli.

**Unitary İşlemler;** Kuantum sistemlerinde durumların evrimini temsil eder ve bu işlemler daima tersinir (invertible) ve norm koruyucu (norm-preserving) olmalı.

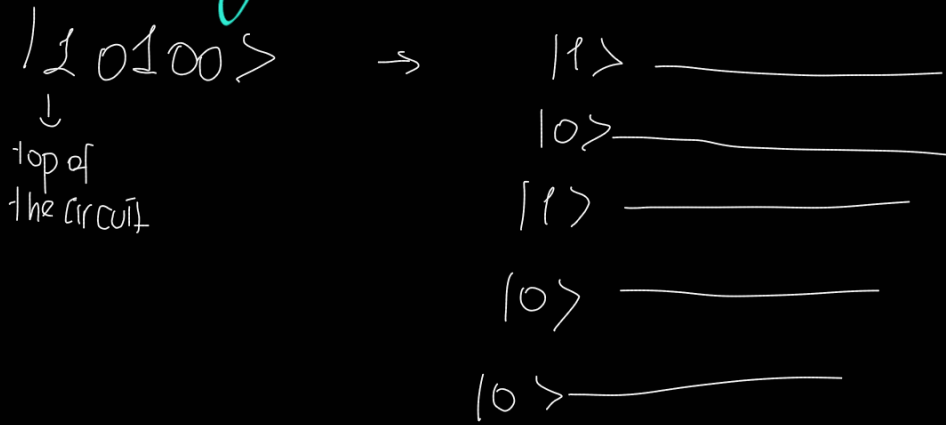
**Tensor Product;** İki kuantum sisteminin birleştirilmesi.

$U^\dagger = U$   
 ↳ matrisin hermiten eşleniği  
 (Hermitian conjugate)

bu yüzden bir kuantum  
 sis. kuantum için kullanılır.

$$|\psi\rangle \otimes |\phi\rangle$$

## Qubit Ordering Convention;



Sev kibrarı'nın kibrarı'p  
 değişir bazından check et.

```
dev = qml.device('default.qubit', wires=3)

@qml.qnode(dev)
def circuit(feature_vector):
    qml.BasisEmbedding(features=feature_vector, wires=range(3))
    return qml.state()

X = [1,1,1]
```

The resulting circuit is:

```
>>> print(qml.draw(circuit, expansion_strategy="device")(X))
0: —X—| State
1: —X—| State
2: —X—| State
```

## Multi Qubit Bases;

$$|10\rangle \Rightarrow 2_2$$

$$|111\rangle = 7_2$$

$$\begin{matrix} \downarrow & \downarrow & \downarrow \\ 2^2 & 2^1 & 2^0 \\ 4 + 2 + 1 = 7 \end{matrix}$$

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |100\rangle$$

Bütün vektörlerde **tek bir 1** var.

Bu vektörlerin bir temel oluşturduğunu aklınıza geçiyor.

$$\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} |10\rangle$$

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

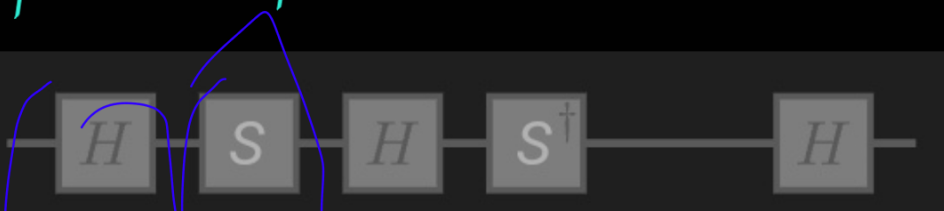
Bu temel vektörlerin doğrusal kombinasyonlarıyla herhangi bir  $4 \times 1$  kompleks değerli vektör oluşturulabilir, yani bu vektörler, dört boyutlu uzayda bir temel oluşturur ve herhangi bir vektör bu baz vektörlerin doğrusal kombinasyonu olarak yazılabilir.

Everytime when a new qubit added we multiple w/ 2 so n qubit systems has  $2^n$  basis vectors w/  $2^n$  elements.  $\rightarrow$  To operate on such basis vectors, we'll need **unitary matrices size of  $2^n \times 2^n$**   $\rightarrow$  This exponential dependence means that simulating quantum com. will become computationally intractable very quickly.

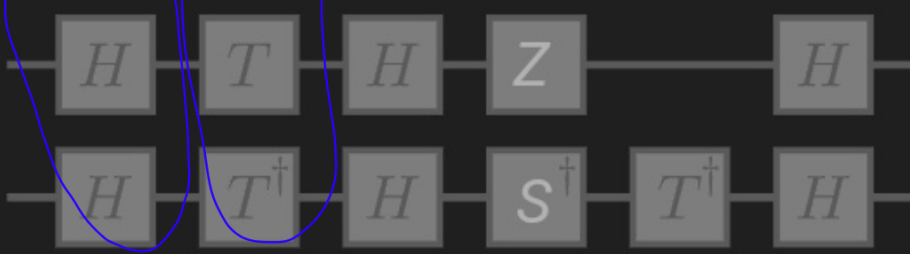
**Exponansiyel Bağımlılık;** Bir sistemin büyüklüğü arttıkça gereken hesaplamalı kaynaklar (zaman, hafıza, işlem gücü) **exponansiyel** olarak  $\nearrow$  yani qubit sayısı arttıkça, klasik bilgisayarlar bu kuantum sistemlerini simüle etmek için giderek daha zor ve zaman alıcı hale gelir.

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

**Separable Operations;**



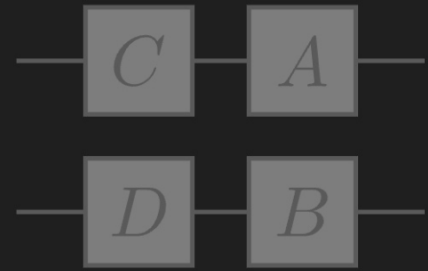
$\rightarrow$  Bu devrenin (circuit) Her bir wire'ını



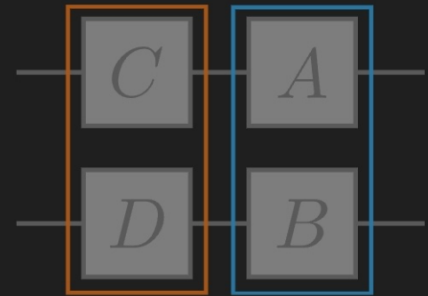
→ (kütümanı) single qubit operation gıbt  
→ dıönelıblr sm.

For example, the first step in the circuit is to apply a Hadamard to each qubit. This corresponds to applying  $H \otimes H \otimes H$ . The second layer applies  $S \otimes T \otimes T^\dagger$ , and so on. While circuits are read from left to right, the matrices are applied to the states from right to left, i.e., the result of applying the first two layers of gates to an input state  $|\psi\rangle$  would be  $(S \otimes T \otimes T^\dagger)(H \otimes H \otimes H)|\psi\rangle$ .

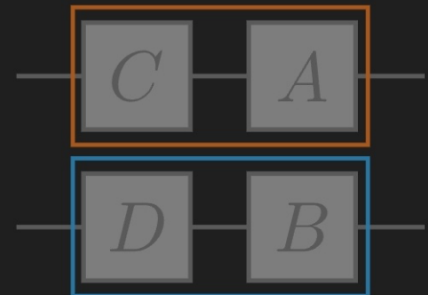
Let's start by drawing what this would look like as a quantum circuit:



We can think of it as  $(A \otimes B) \cdot (C \otimes D)$  like so:



However, since these are separable operations, the qubits are acted on independently of each other, so we could just as well conceptually group the operations together like so:



Each qubit has two unitary operations acting on it; applying  $C$  then  $A$  to the first qubit is equivalent to applying the combined operation  $AC$ , and similarly for the second qubit. Thus, we can rewrite this circuit as



**Exercise 1.11.4.** Suppose we have two tensor product operations,  $A \otimes B$ , and  $C \otimes D$ . A cool fact is that if we multiply these two together, the multiplication works on each side of the tensor product independently, i.e.,  $(A \otimes B) \cdot (C \otimes D) = (AC) \otimes (BD)$ . If you like, you can take a crack at proving this mathematically. However, there is a more intuitive way of understanding this: using a circuit diagram and your knowledge of how single- and two-qubit operations work, reason why this identity must hold.

▼ Hint.

Start by drawing the quantum circuit that applies  $A \otimes B$  followed by  $C \otimes D$ , then look at the operations acting on the individual qubits.

Use PennyLane to create the state  $|+1\rangle = |+\rangle \otimes |1\rangle$ . Then, return two measurements:

- the expectation value of  $Y$  on the first qubit
- the expectation value of  $Z$  on the second qubit

In PennyLane, you can return measurements of multiple observables as a tuple, as long as they don't share wires.

```
import pennylane as qml
from pennylane import numpy as np

# İki kubitli bir cihaz oluřturur
dev = qml.device('default.qubit', wires=2)

@qml.qnode(dev)
def two_qubit_circuit():
    # |+>|1> durumunu hazırlar
    qml.Hadamard(wires=0)
    qml.PauliX(wires=1)

    # İki beklenen değeri döndür: ilk kubit için Y, ikinci kubit için Z
    k = qml.expval(qml.PauliY(wires=0))
    m = qml.expval(qml.PauliZ(wires=1))
    return [k, m]  # Beklenen değeler k ve m değışiklik atar ve liste olarak döndürölür.
```

```
# Devreyi çalıştır ve sonuçları yazdır
print(two_qubit_circuit())
```

```
print(two_qubit_circuit())
```

```
[tensor(0., requires_grad=True), tensor(-1., requires_grad=True)]
```

Bu çıktıyı iki tensor nesnesi var. Her biri beklenen değeri (expectation value) temsil ediyor.

1. İlk tensor  
İlk kubit (wire-0) üzerinde Pauli-Y operatörünün beklenen değerini temsil eder.  
requires\_grad=True → Bu tensorun geri yayılım/backpropagation için gradyan hesaplamaları gerektiğini belirtir. → Kuantum devreleri optimize etmeye ve nümerik öğrenme uygulamalarında yararlıdır.

## 2. İkinci Tensor (tensor(-1., requires\_grad=True))

- Bu, ikinci kubit (wire 1) üzerinde Pauli-Z operatörünün beklenen değerini temsil eder.
- Beklenen değerin (-1.0) olduğunu gösteriyor.
- Yine, requires\_grad=True ifadesi, bu tensorun gradyan hesaplamaları için uygun olduğunu belirtir.

## Geri Yayılım (Backpropagation)

Geri yayılım, bir yapay sinir ağında hatayı minimize etmek için kullanılan bir algoritmadır. Bu algoritma, ağın çıktısı ile hedef değerler arasındaki hatayı hesaplar ve bu hatayı azaltmak için ağın ağırlıklarını geri yayarak günceller. Bu süreç, gradyan inişi (gradient descent) algoritmasının bir parçasıdır.

## Gradyan Hesaplamaları

Gradyan, bir fonksiyonun değişim oranını gösteren vektördür. Gradyan hesaplamaları, fonksiyonun minimum veya maksimum noktalarını bulmak için kullanılır. Geri yayılımda, gradyanlar hata fonksiyonunun türevlerini temsil eder ve bu türevler, ağın ağırlıklarını güncellemek için kullanılır.



which we can now see to be the tensor product  $AC \otimes BD$ . □

Pauli-Y operatörü bu süperpozisyon durumunda sıfır beklenen değerine sahiptir.

Pauli-Z operatörünün beklenen değeri (-1.0) olur çünkü Pauli-Z operatörü  $|1\rangle$  durumunda (-1) beklenen değerine sahiptir.

Her iki tensorun da requires\_grad=True olması, bu tensorların gradyan hesaplamaları için uygun olduğunu ve potansiyel olarak optimizasyon süreçlerinde kullanılabileceğini belirtir.

## Kuantum Hesaplama Geri Yayılım ve Gradyan Hesaplamaları

Kuantsal devrelerde, özellikle parametrik kuantum devrelerinde, parametreleri optimize etmek için geri yayılım ve gradyan hesaplamaları kullanılabilir. Bu optimizasyon, kuantum devrelerinin belirli bir hedef fonksiyonu minimize edecek şekilde ayarlanmasını sağlar.

requires\_grad=True

Bu ifade, bir tensorun gradyan hesaplamaları için uygun olduğunu belirtir. PennyLane ve benzeri kuantum hesaplama kütüphanelerinde, bu ifade ile bir değişkenin veya tensorun optimizasyon süreçlerinde kullanılabileceğini belirtiriz.

# Expectation Value of two qubit Observable

Write a PennyLane circuit that creates the state  $|1-\rangle = |1\rangle \otimes |-\rangle$ . Then, measure the expectation value of the *two-qubit observable*  $Z \otimes X$ . In PennyLane, you can combine observables using the @ symbol to represent the tensor product, e.g., `qml.PauliZ(0) @ qml.PauliZ(1)`.

```
import pennylane as qml
```

```
dev = qml.device("default.qubit", wires=2)
```

```
@qml.qnode(dev)
```

```
def create_one_minus():
```

```
#####
```

```
# YOUR CODE HERE #
```

```
#####
```

```
# PREPARE  $|1\rangle|-\rangle$ 
```

```
qml.PauliX(wires=0)
```

```
qml.PauliX(wires=1)
```

```
qml.Hadamard(wires=1)
```

```
# RETURN A SINGLE EXPECTATION VALUE  $Z \otimes X$ 
```

```
return qml.expval(qml.PauliZ(0) @ qml.PauliX(1))
```

→ Bu ifade, bir tensorun gradyan hesaplamaları için uygun olduğunu belirtir.

```
print(create_one_minus())
```

Implement the following circuit twice. For one version, measure the observables  $Z$  on the first qubit (i.e.,  $Z \otimes I$ ), and  $Z$  on the second qubit ( $I \otimes Z$ ). For the other version, measure the observable  $Z \otimes Z$ . How do you think the results of the first circuit will relate to those of the second? Plot the results as a function of  $\theta$  to test your hypothesis.



*Tip.* In PennyLane, you don't need to specify the identity portion of observables. For example,  $I \otimes Z$  is simply `qml.PauliZ(1)` rather than `qml.Identity(0) @ qml.PauliZ(1)`.

▼ *Hint.*

Consider how the observables themselves are related. If you have  $Z \otimes I$  and  $I \otimes Z$ , can you produce  $Z \otimes Z$ ?

→ Örneğin  $Z \otimes I$  ve  $I \otimes Z$  ölçüm sonuçları  $Z \otimes Z$  ölçüm sonucunu hesaplıyorsun ya  $\cos$  0 yu?den? Bu devrede, ölçüm sonucunu döndürür. Bu, tek bir ölçüm yapmaktan daha güvenilir ve istatistik olarak anlamlı sonuçlar elde etmeni sağlar.

→ Her iki qubit için Pauli-Z operatörünün belirlenen değerlerini döndürür.

→ Bu fonksiyon Circuit\_1 devresinden elde edilen  $Z_1$  ve  $Z_2$  sonuçlarını alır ve bu sonuçları çarparak  $Z \otimes Z$  sonucuna eşdeğer den sonuçları üretir.

## All Tied Up

## Mathematics of Entangled State:

Ayrıkabilir Çoklu Qubit Operasyonları; Başlangıçta, çoklu qubit sistemlerinde tartışılan operasyonlar ayrılabilirdir. Yani her qubit, diğer qubitlerle etkileşime girmeyen tek qubit kapıları tarafından işlenir. Bu her qubit'in durumunun diğerlerinden bağımsız olarak değiştirildiği en basit çoklu qubit işlemidir.

## Dolanıklık

İki veya daha fazla qubit'in quantum durumu birbirleriyle bağımlı hale gelir  $\rightarrow$  Bir qubit'in durumu mesafeye bağımsızlığın anında diğer qubit'in durumunu etkilemektedir. Bu yerel olmayan korelasyon, klasik sistemlerin kopyalanamayacağı temel bir özelliktir.

**Kuantum Bilgi Teleportasyonudur;** Burada dolanıklık, kubitler arasında kuantum bilgisini fiziksel olarak tasımadan transfer etmek için kullanılır.

**Non-local correlation;** Kuantum mekaniğinde birbirleriyle dolanık olan parçacıklar arasında meydana gelen ve bu parçacıklar arasındaki mesafeye bakılmaksızın anında gerçekleşen bağlantıyı ifade eder.

### Yerel Korelasyon vs Yerel Olmayan Korelasyon

- **Yerel Korelasyon:** Bu tür korelasyon, klasik sistemlerde gözlemlenir. İki parçacık arasındaki ilişki, doğrudan fiziksel temasa veya yakın mesafeye dayanır. Örneğin, bir çift zar attığınızda, bir zarın sonucu diğerinden bağımsızdır ve herhangi bir korelasyon varsa, bu korelasyon her iki zarın atılış şekli ve yüzey özellikleri gibi yerel etkileşimlerden kaynaklanır.
- **Yerel Olmayan Korelasyon (Non-local Correlation):** Bu, kuantum mekaniğinde gözlemlenen ve parçacıkların birbirinden ne kadar uzakta olduğuna bakılmaksızın geçerli olan korelasyondur. İki dolanık parçacık, birbirinden çok uzak mesafelerde bile olsa, birinin durumu anında diğerinin durumunu etkileyebilir. Bu, Einstein'ın "spooky action at a distance" (mesafede ürkütücü etki) olarak adlandırdığı olgudur. Bu tür korelasyonlar, klasik fiziğin öngörleriyle uyumsuzdur ve sadece kuantum dolanıklık ile açıklanabilir.

1. **Bell Durumları:** İki kubitin dolanık olduğu ve bu kubitlerin birinde yapılan bir ölçümün diğer kubitin durumunu belirlediği özel kuantum durumlarıdır.
2. **Kuantum Teleportasyonu:** Bir parçacığın kuantum durumunun başka bir parçacığa aktarılması sürecidir, burada dolanıklık kullanılarak bilgi mesafe tanımaksızın anında iletilir.

Bu yerel olmayan korelasyon, kuantum bilgisayarlarının ve kuantum bilgi işlem teknolojilerinin temelini oluşturur ve klasik sistemlerin kopyalayamayacağı bir avantaj sağlar.